

ROS Robot URDF Model Instruction

1. Getting Ready

To grasp the URDF model, check out "Lesson 1 URDF Model Introduction" for the key syntax. This part quickly breaks down the robot model code and its components.

2. Check Code of Robot Model

- 1) Start the robot, and access the robot system desktop using VNC.



- 2) Click-on  to create a command-line terminal.

- 3) Run the command and hit Enter to disable the app auto-start service.

```
~/stop_ros.sh
```

```
> ~/.stop_ros.sh
```

- 4) Execute the command and hit Enter key to navigate to the folder containing startup programs.

```
colcon_cd mentorpi_description
```



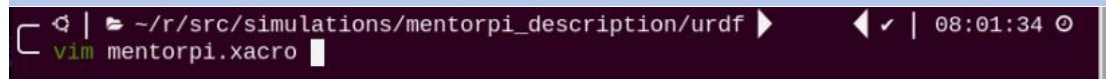
- 5) Execute the command to navigate to the robot simulation model folder.

```
cd urdf
```

```
> cd urdf
```

- 6) Execute the command to access the robot simulation model folder.

```
vim mentorpi.xacro
```



- 7) Locate the code below:

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <robot name="rosmmentor" xmlns:xacro="http://ros.org/wiki/xacro">
3   <xacro:property name="machine" value="$(env MACHINE_TYPE)"/>
4   <xacro:if value="${machine} == 'ROSMentor_Mecanum'">
5     <xacro:include filename="$(find rosmmentor_description)/urdf/mecanum.xacro"/>
6   </xacro:if>
7   <xacro:if value="${machine} == 'ROSMentor_Acker'">
8     <xacro:include filename="$(find rosmmentor_description)/urdf/ack.xacro"/>
9   </xacro:if>
10  <xacro:include filename="$(find rosmmentor_description)/urdf/inertial_matrix.xacro" />
11 </robot>

```

Determine the current form of the robot by reading the machine variable to load the corresponding URDF model .

Multiple URDF models are called to compose a complete robot.

File Name	Device
inertial_matrix	Inertia matrix
mecanum	Mecanum urdf model
ack	Ackerman urdf model

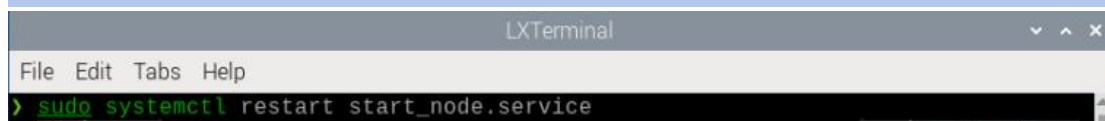
After experiencing the Lidar game, you can activate the app service either by using a command or restarting the robot. If the app service is not activated, related app functions will be disabled. In the case of a robot restart, the app service will start automatically.



Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

Note: please enter the command in the system path, not in the Docker container.

sudo systemctl restart start_node.service



3. Brief Analysis of Robot's Main Body Model

MentorPi has two types of chassis: Mecanum wheel chassis and Ackerman chassis. Here, we'll focus on the Mecanum wheel chassis. Although the main model files for the Ackerman chassis are mostly the same, any differences will be explained later in this article.

Open a new command terminal. Enter the command to access the robot model file, which contains the description of each part of the robot model.

```
vim mecanum.xacro
```

```
vim mecanum.xacro
<?xml version="1.0" ?>
<robot name="mecanum" xmlns:xacro="http://ros.org/wiki/xacro">
```

This is the beginning of the URDF file. It specifies the XML version and encoding, and defines a robot model named "mecanum". The xmlns:xacro namespace is utilized here to generate URDF using Xacro macro definitions. The following line of code defines a Xacro property named "black", which means the black material.

```
<material name="black">
  <color rgba="0 0 0 1"/>
</material>
```

A joint called "imu_link" is defined with a type of "fixed", indicating it's a stationary joint. It connects the parent link "base_link" with the child link "imu_link".

Then, the values of the joint's origin and axis are defined.

```
<link name="imu_link"/>
<joint name="imu_joint" type="fixed">
  <parent link="base_link"/>
  <child link="imu_link"/>
  <origin xyz="0 0 0" rpy="0 0 -1.57"/>
</joint>
```

The link named "base_footprint" is defined as the robot's chassis.

```
<link name="base_footprint"/>
<joint name="base_footprint_to_base_link" type="fixed">
  <parent link="base_footprint"/>
  <child link="base_link"/>
```

```
<origin xyz="0 0 0.07" rpy="0 0 0"/>
</joint>
```

The following code defines a link named “base_link”, including its inertial, visual, and collision properties.

The <inertial> section specifies the link's inertial properties, like mass and inertia. It includes the <origin> tag, indicating the position and orientation of the inertial coordinate system relative to the link's coordinate system. The <mass> tag denotes the link's mass, and the <inertia> tag defines its moment of inertia about its principal axis.

The <visual> section determines the visual representation of the link. It includes the <origin> tag for the visualization coordinate system's position and orientation relative to the link. The <geometry> tag defines the visual shape, here a grid, while the <mesh> tag specifies the file name of the mesh representing the link's visual appearance. Lastly, the <material> tag specifies the color or texture. The “base_link.STL” defines the material.

The <collision> section specifies the link's collision properties. It resembles the <visual> section but is for collision detection, not visualization. It includes the <origin> and <geometry> tags for defining position, direction, and shape of the collision representation.

In summary, this code defines a link in the robot model, detailing its inertial, visual, and collision properties. In simulations or visualizations, the mesh files specified in the <visual> and <collision> sections are utilized for visual representation and collision detection with other links.

```
<link name="base_link">
  <inertial>
    <origin rpy="0 0 0" xyz="7.82634169708696E-05 3.66518389137982E-05
0.0068167426737893"/>
    <mass value="0.080358044299185"/>
```

```

    <inertia ixx="9.92047502462735E-05" ixy="3.20099144678921E-08"
ixz="1.15869166655817E-05" iyy="0.000292461515602123"
iyz="-2.92890751882614E-08" izz="0.000367604260834833"/>

  </inertial>

  <visual>

    <origin rpy="0 0 0" xyz="0 0 0"/>

    <geometry>

      <mesh
filename="package://rosmotor_description/meshes/mecanum/base_link.STL"
/>

    </geometry>

    <material name="">

      <color rgba="1 1 1 1"/>

    </material>

  </visual>

  <collision>

    <origin rpy="0 0 0" xyz="0 0 0"/>

    <geometry>

      <mesh
filename="package://rosmotor_description/meshes/mecanum/base_link.STL"
/>

    </geometry>

  </collision>

</link>

```

The following code describes a link named “wheel_lf_Link”, which represents the left front wheel of the robot.

The <inertial> tag describes the inertial properties of the link, including its mass and moments of inertia. The mass of the link is set to 0.0378743634664218, and the values of its moments of inertia are specified.

The <visual> tag defines the visual appearance of the link, including its geometry and color. The visual appearance of the link uses an STL mesh model located at

“package://rosmentor_description/meshes/mecanum/wheel_lf_Link.STL”. The color is set to white with an RGBA value of 1 1 1 1, indicating a completely opaque white color.

The <collision> tag defines the collision properties of the link for collision detection. The collision model also uses an STL mesh model located at “package://rosmentor_description/meshes/mecanum/wheel_lf_Link.STL”

```
<link name="wheel_lf_Link">

  <inertial>

    <origin rpy="0 0 0" xyz="6.23194347770945E-05 -0.00574614971695361
2.98597149463314E-05"/>

    <mass value="0.0378743634664218"/>

    <inertia ixx="3.67678518315829E-06" ixy="-1.42189561270372E-09"
ixz="-1.20320324868622E-08" iyy="5.81008995754212E-06"
iyz="1.72329758812212E-09" izz="3.67939460898329E-06"/>

  </inertial>

  <visual>

    <origin rpy="0 0 0" xyz="0 0 0"/>

    <geometry>

      <mesh
filename="package://rosmentor_description/meshes/mecanum/wheel_lf_Link.
STL"/>

    </geometry>

    <material name="">

      <color rgba="1 1 1 1"/>

    </material>

  </visual>
```

```
<collision>

  <origin rpy="0 0 0" xyz="0 0 0"/>

  <geometry>

    <mesh

filename="package://rosmotor_description/meshes/mecanum/wheel_lf_Link.
STL"/>

    </geometry>

  </collision>

</link>
```

The following is the description of the joint:

```
<joint name="wheel_lf_Joint" type="continuous">

  <origin rpy="0 0 0" xyz="0.067052 0.07591 -0.018408"/>

  <parent link="base_link"/>

  <child link="wheel_lf_Link"/>

  <axis xyz="0.00012228 1 -4.0642E-05"/>

</joint>
```

The above code defines a joint named “wheel_lf_Joint” for the left front wheel. The type of this joint is “continuous”, indicating that it is a continuously rotating joint used to describe a component that can rotate without limits, such as a wheel.

The <origin> tag specifies the position and orientation of the joint by “xyz=“0.067052 0.07591 -0.01”.

The parent and child links of the joint are specified using the <parent> and <child> tags, respectively. The parent link is “base_link”, and the child link is “wheel_lf_Link”, indicating that the joint connects the base link and the left front wheel link. The <axis> tag is used to define the rotation axis of the joint, with the direction specified by the xyz=“0.00012228 1 -4.0642E-05” attribute.

This code describes a joint in a URDF file that connects the parent link

“base_link” to the child link “wheel_Lf_Link”. This joint describes the rotational connection of the left front wheel in the robot model, allowing it to rotate infinitely around the specified axis.